

OptiCrop

Smart Agricultural Production Optimization Engine

Neelima | B.Tech AI & Data Science | VRSEC, Vijayawada

Abstract

OptiCrop is a machine learning-powered smart agricultural production optimization engine designed to assist farmers, agricultural researchers, and policymakers in making data-driven crop selection decisions. The system accepts seven key soil and environmental parameters — Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall — and uses a trained Random Forest classifier to recommend the most suitable crop for cultivation under those exact conditions.

The project was developed using Python-based data science libraries including NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn, and deployed as a three-page web application using the Flask framework. The model achieved 99.55% test accuracy after comparing Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest classifiers on the Crop Recommendation Dataset (2,200 samples, 22 balanced crop classes).

Project Statement

OptiCrop is an AI-powered smart agricultural recommendation system that uses machine learning to recommend the most suitable crop for a given set of soil and climate conditions. By analysing Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, soil pH, and rainfall, the system replaces guesswork in crop selection with a fast, measurable, data-driven recommendation — helping farmers, agricultural researchers, and policymakers make better farming decisions.

The system analyses environmental and soil data, compares multiple machine learning classifiers, deploys the best-performing model (Random Forest, 99.55% test accuracy), and exposes it through a simple Flask web application accessible from any browser.

Features

- Smart Crop Recommendation — Enter N, P, K, temperature, humidity, pH, and rainfall to instantly receive the most suitable crop recommendation powered by a trained Random Forest model.

- **Multi-Algorithm Comparison** — Four supervised classifiers (Logistic Regression, KNN, Decision Tree, Random Forest) and K-Means Clustering are trained and compared, with the best model selected for deployment.
- **High Accuracy Prediction** — Random Forest achieves 99.55% test accuracy across all 22 crop classes.
- **Exploratory Data Analysis** — Univariate, bivariate, and multivariate analysis provide deep insight into crop-environment relationships.
- **Outlier Detection and Handling** — IQR-based detection on the Potassium feature with log transformation.
- **Seasonal Crop Classification** — Crops are automatically grouped into Winter, Summer, Rainy, and Moderate seasons based on their average climate profiles.
- **Simple Web Interface** — A clean, responsive three-page Flask application usable by anyone with no ML background.
- **Fully Reproducible Pipeline** — Dataset, model artefacts, training notebook, and application code all included in the repository.

Scenarios

Scenario 1: A farmer enters soil nutrient measurements and local climate data — Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall — into the Find Your Crop form. The system analyses the conditions and returns the most suitable crop for cultivation, helping the farmer make a data-driven planting decision rather than relying on tradition or informal advice.

Scenario 2: An agricultural researcher uses the same system to explore which crops are best suited to a range of environmental profiles. By submitting multiple combinations of soil and climate parameters, the researcher can study crop-environment relationships and compare the model's recommendations against known agricultural science.

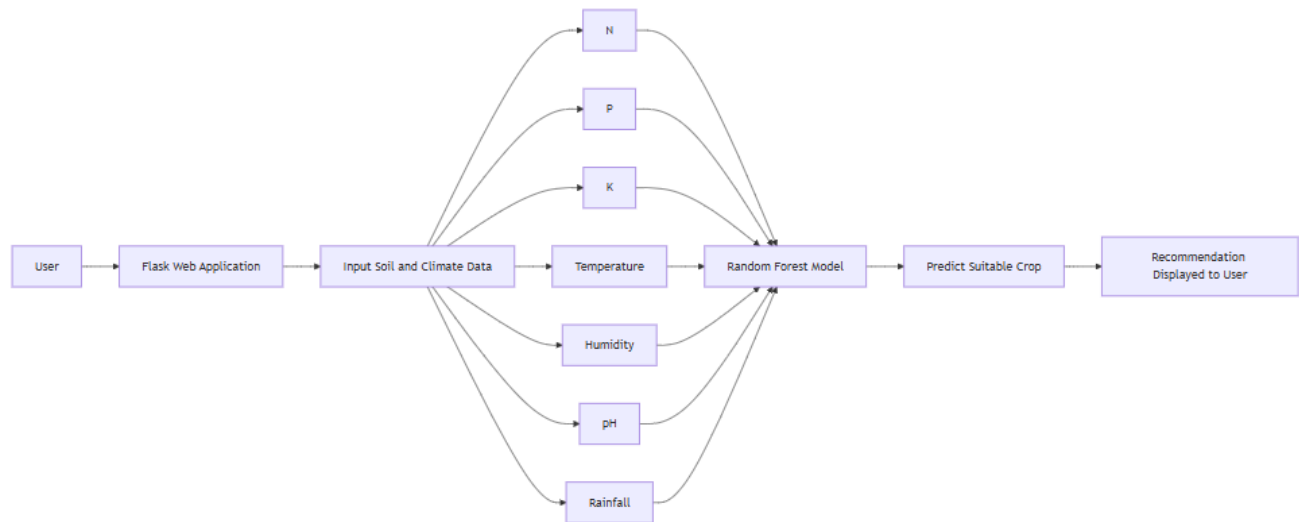
Scenario 3: A policymaker or agribusiness stakeholder uses OptiCrop's underlying model and dataset insights (correlation heatmaps, cluster analysis, model comparison table) to understand regional crop suitability patterns, supporting evidence-based decisions around resource allocation, subsidy planning, and sustainable farming strategy.

Technical Architecture

OptiCrop uses a modular three-tier architecture to deliver fast, ML-driven crop recommendations.

- **Presentation Layer:** Three HTML pages (Home, About, Find Your Crop) styled with Bootstrap 5, served as Flask templates.
- **Application Layer:** A Flask backend with three routes (/, /about, /find-your-crop) handling form rendering and prediction requests via POST.
- **Machine Learning Layer:** A StandardScaler, trained Random Forest classifier, and LabelEncoder — all serialised with pickle and loaded once at app startup for fast inference.
- **Data Layer:** The Crop_recommendation.csv training dataset and the three .pkl model artefacts (model.pkl, scaler.pkl, label_encoder.pkl).

Request flow: Browser → Flask route → input validation → StandardScaler.transform() → RandomForestClassifier.predict() → LabelEncoder.inverse_transform() → rendered HTML response.



Pre-requisites

- **Python 3.x:** <https://www.python.org/downloads/>
- **NumPy:** <https://numpy.org/doc/stable/>
- **Pandas:** <https://pandas.pydata.org/docs/>
- **Scikit-learn:** <https://scikit-learn.org/stable/>
- **Matplotlib:** <https://matplotlib.org/stable/>
- **Seaborn:** <https://seaborn.pydata.org/>
- **SciPy:** <https://docs.scipy.org/doc/scipy/>
- **Flask:** <https://flask.palletsprojects.com/>
- **Dataset:** <https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine>
- **VS Code:** <https://code.visualstudio.com/>

Team Member Details

This project was completed as a solo submission with mentor approval. The original team was inactive.

Name	Institution	Programme	Role	GitHub
Neelima	VRSEC, Vijayawada	B.Tech AI & Data Science	Solo Developer	neelimavana

Project Workflow

Day / Milestone	Focus Area
Day 1	Environment setup, repo creation, dataset download and loading
Day 2	Exploratory Data Analysis — univariate, bivariate, multivariate
Day 3	Data pre-processing — null check, outlier handling, train/test split
Day 4	Model building — K-Means clustering + 4 classifiers trained and compared
Day 5	Model finalisation — best model saved as .pkl artefacts
Day 6	Flask application built — 3-page web app working end-to-end
Day 7	Testing, documentation, GitHub push, demo recording, submission

Milestone 1: Data Collection and EDA

- **Activity 1.1:** Download the Dataset
- **Activity 1.2:** Import Libraries
- **Activity 1.3:** Read and Explore the Dataset
- **Activity 1.4:** Univariate Analysis
- **Activity 1.5:** Bivariate Analysis
- **Activity 1.6:** Multivariate Analysis

Milestone 2: Data Pre-Processing

- **Activity 2.1:** Check for Null Values
- **Activity 2.2:** Handling Outliers
- **Activity 2.3:** Extracting Seasonal Crops
- **Activity 2.4:** Train/Test Split

Milestone 3: Model Building

- **Activity 3.1:** K-Means Clustering
- **Activity 3.2:** Supervised Model Training and Comparison
- **Activity 3.3:** Saving the Best Model
- **Activity 3.4:** Predict the Best Crop Given Parameters

Milestone 4: Application Building

- **Activity 4.1:** Building HTML Pages
- **Activity 4.2:** Building the Python Backend (app.py)
- **Activity 4.3:** Run and Test the Application

Milestone 5: Conclusion

Milestone 1: Data Collection and EDA

Activity 1.1: Download the Dataset

The Crop Recommendation Dataset is downloaded from Kaggle and saved as data/Crop_recommendation.csv. The dataset contains 2,200 samples, 22 crop classes (100 samples each), and 7 features: N, P, K, temperature, humidity, ph, rainfall.

[Dataset Link](#)

ATHARVA INGLE · UPDATED 6 YEARS AGO

643<> CodeDownload

Crop Recommendation Dataset

Maximize agricultural yield by recommending appropriate crops



[Data Card](#) [Code \(263\)](#) [Discussion \(13\)](#) [Suggestions \(2\)](#)

About Dataset

Context

Precision agriculture is in trend nowadays. It helps the farmers to get informed decision about the farming strategy. Here, I present you a dataset which would allow the users to build a predictive model to recommend the most suitable crops to grow in a particular farm based on various parameters.

Usability ⓘ
7.06

License
[Apache 2.0](#)

Expected update frequency
Not specified

Activity 1.2: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans
plt.style.use('fivethirtyeight')
```

Activity 1.3: Read and Explore the Dataset

```
df = pd.read_csv('../data/Crop_recommendation.csv')
df.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

df.shape # (2200, 8)

df.info()

```
(2200, 8)
<class 'pandas.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   N                2200 non-null   int64
1   P                2200 non-null   int64
2   K                2200 non-null   int64
3   temperature      2200 non-null   float64
4   humidity          2200 non-null   float64
5   ph               2200 non-null   float64
6   rainfall          2200 non-null   float64
7   label            2200 non-null   str
dtypes: float64(4), int64(3), str(1)
memory usage: 137.6 KB
```

df.describe()

df.describe()							
✓ 0.0s							
	N	P	K	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

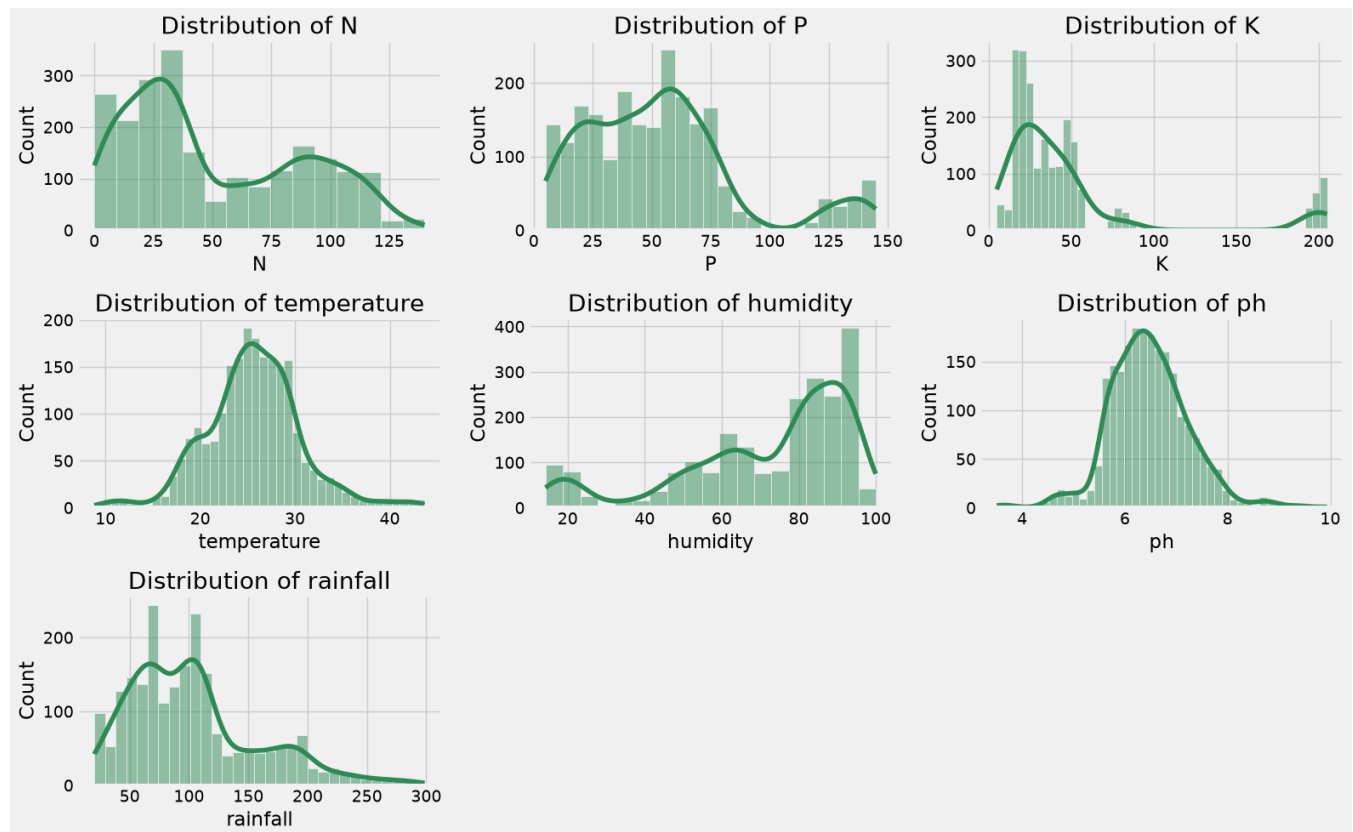
df.isnull().sum() # all zeros — no missing values

```
df.isnull().sum()
```

```
N      0
P      0
K      0
temperature  0
humidity  0
ph      0
rainfall  0
label   0
dtype: int64
```

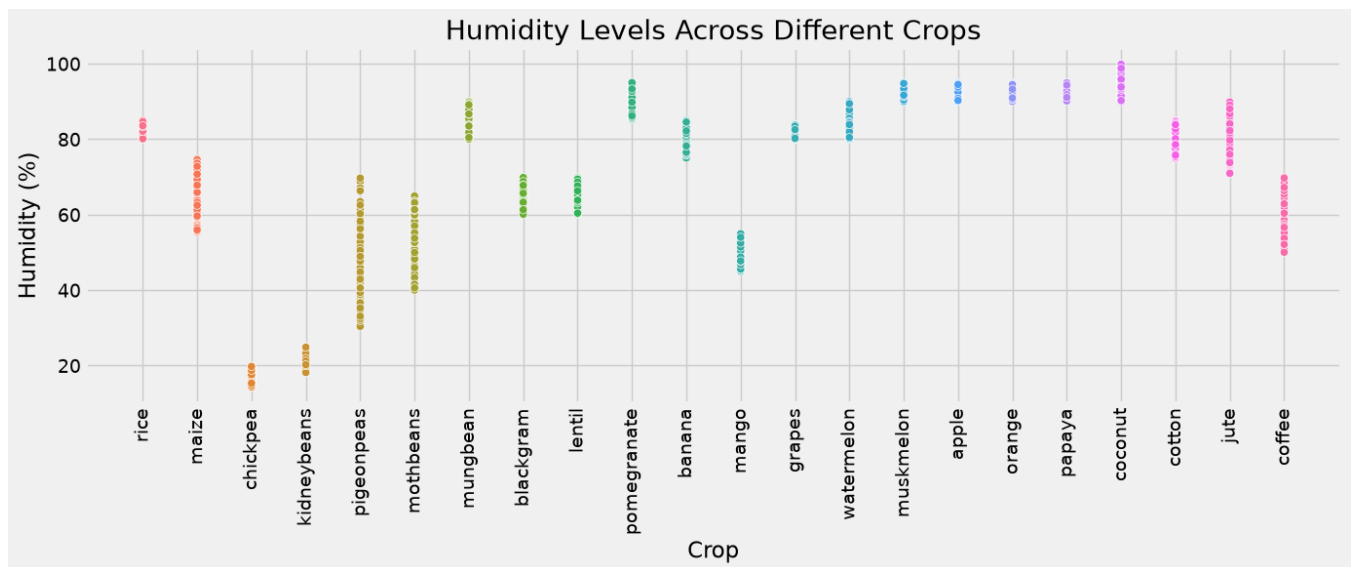
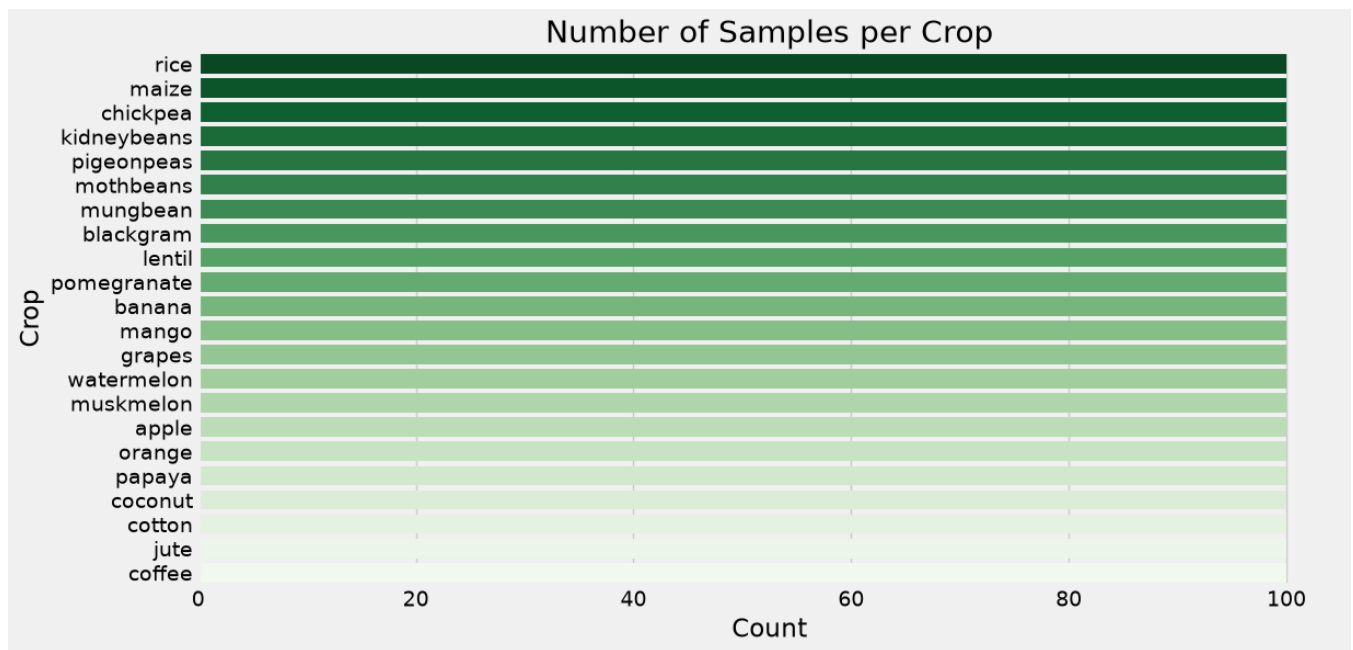
Activity 1.4: Univariate Analysis

Distribution histograms (with KDE) were plotted for all 7 numeric features to understand the spread, skew, and range of each variable. A countplot confirmed perfectly balanced classes — 100 samples per crop.



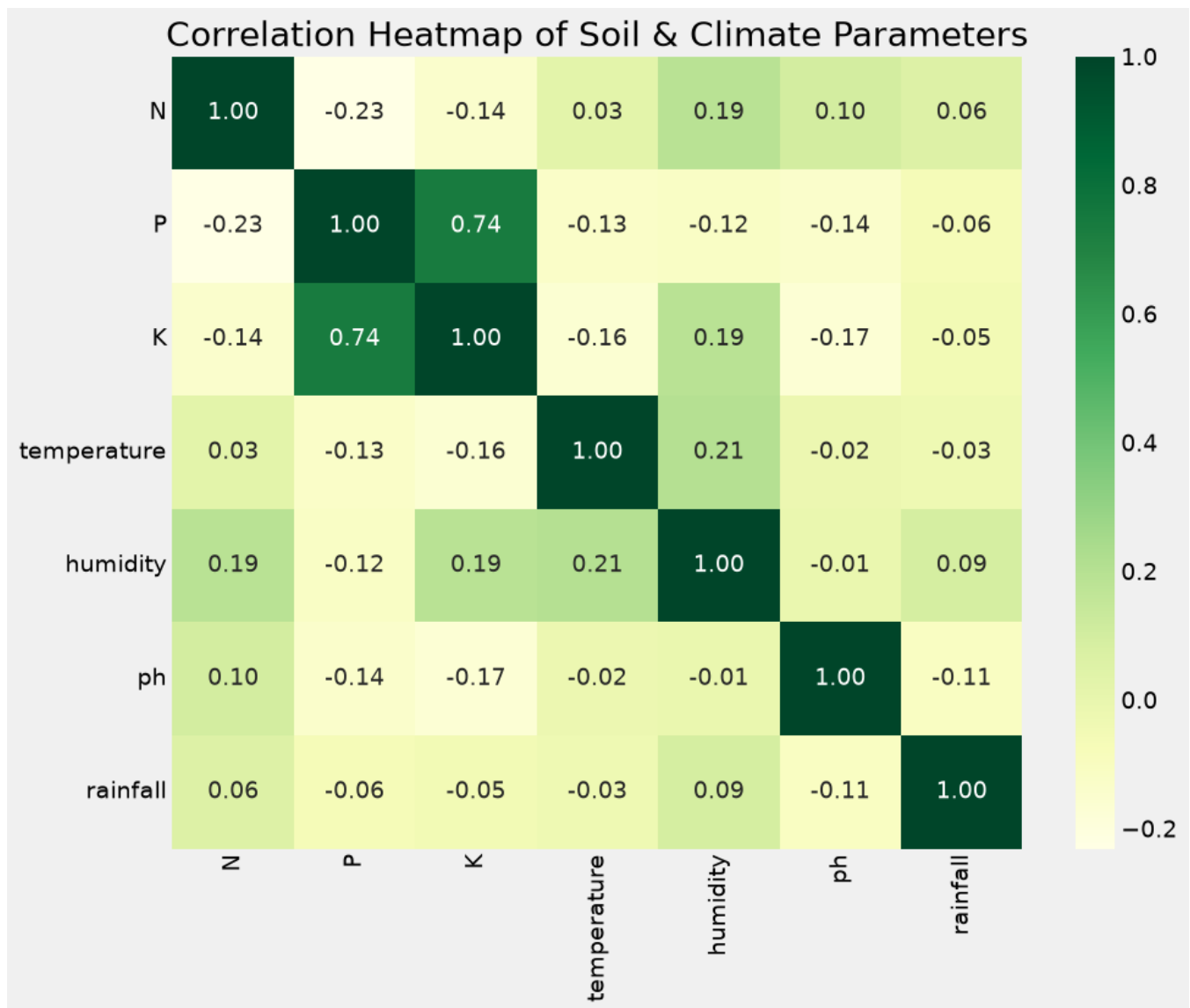
Activity 1.5: Bivariate Analysis

A scatter plot of humidity vs. crop label revealed that crops cluster into distinct humidity bands — for example, rice and jute cluster at high humidity (>80%), while chickpea and lentil cluster at lower values (~15-20%). This tight clustering is a key signal exploited by the model.



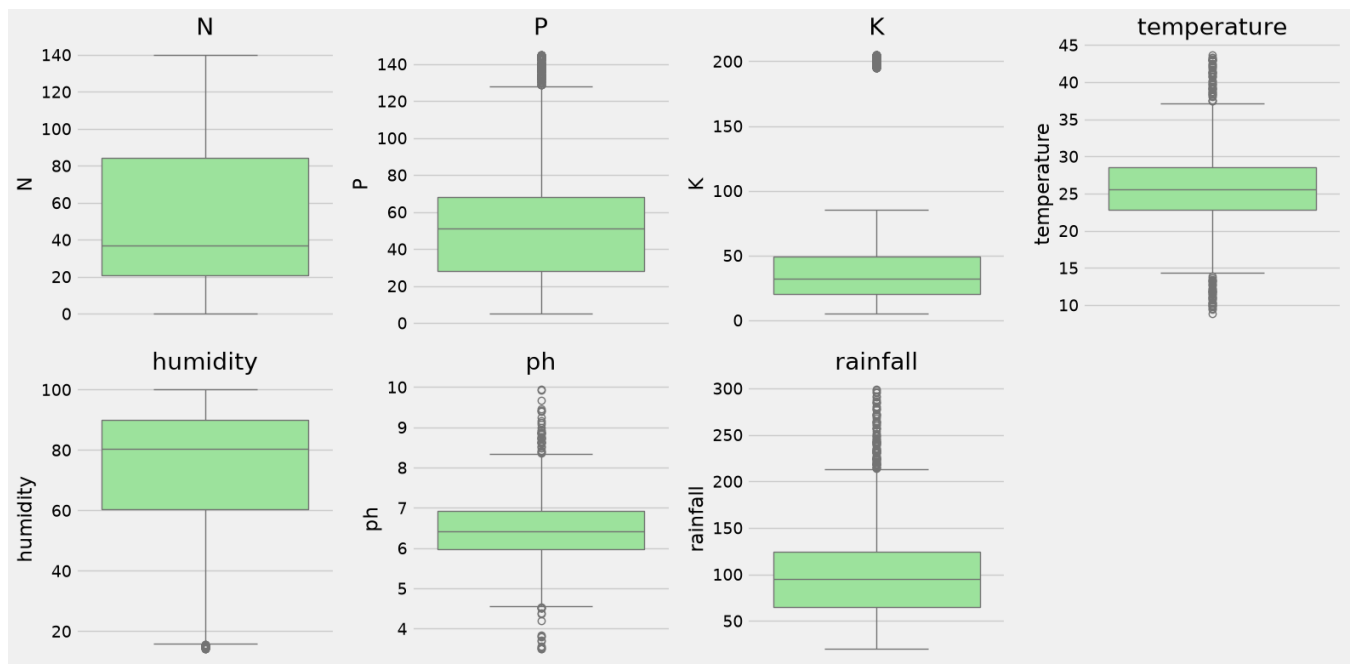
Activity 1.6: Multivariate Analysis

A correlation heatmap of all 7 numeric features was generated to identify inter-feature relationships. A pairplot explored all pairwise combinations. Notable finding: N and K show a moderate positive correlation, while most other feature pairs are largely independent — confirming that all 7 features contribute meaningful, non-redundant information to the model.

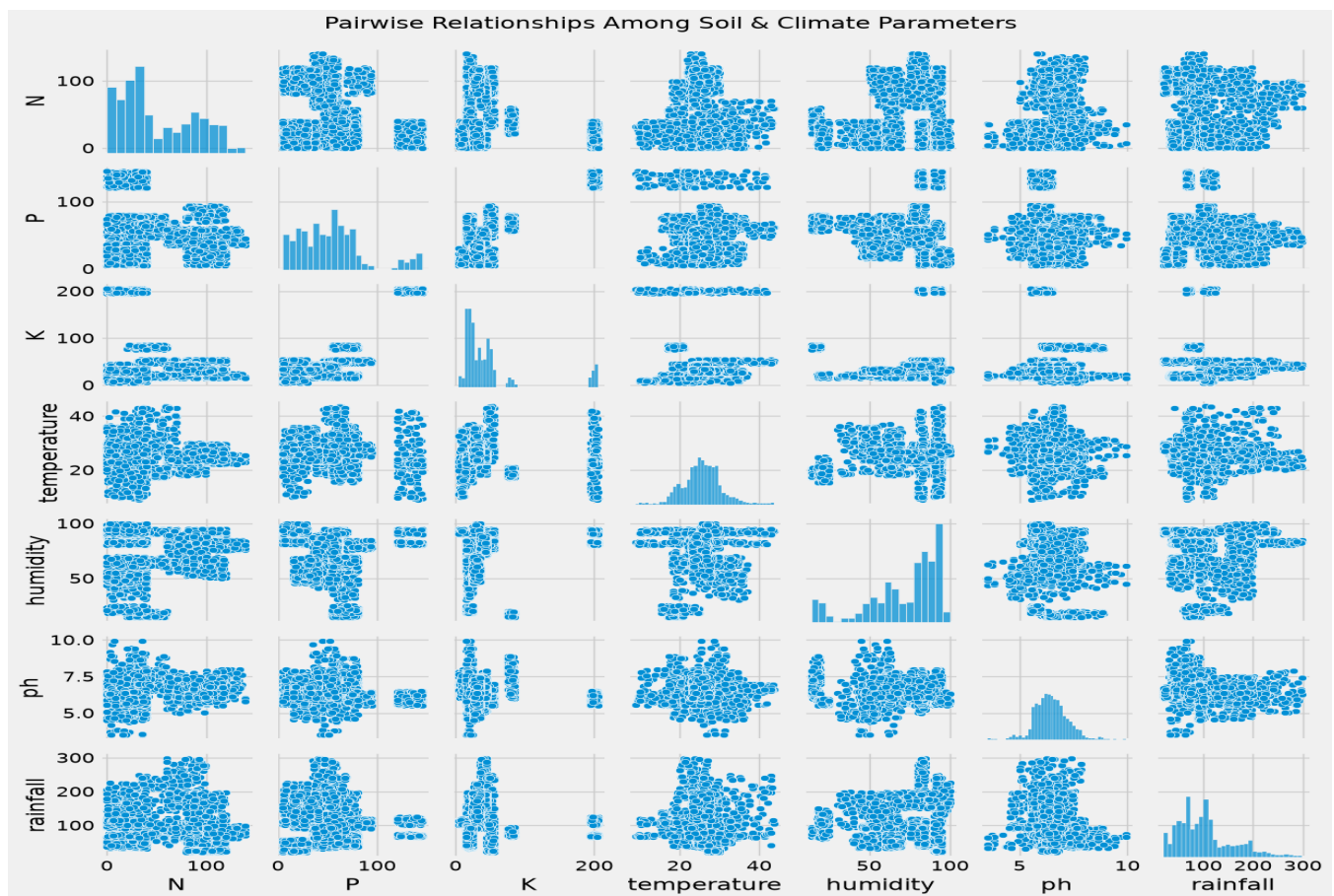


```
plt.figure(figsize=(16, 8))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(2, 4, i)
    sns.boxplot(y=df[col], color='lightgreen')
    plt.title(col)
plt.tight_layout()
plt.show()
```

BOX PLOT



```
sns.pairplot(df[numeric_cols], diag_kind='hist', height=1.8)
plt.suptitle('Pairwise Relationships Among Soil & Climate Parameters', y=1.02)
plt.show()
```



Milestone 2: Data Pre-Processing

Activity 2.1: Check for Null Values

`df.isnull().sum()` confirmed zero missing values across all 8 columns — no imputation required.

```
df.isnull().sum()

N          0
P          0
K          0
temperature 0
humidity    0
ph          0
rainfall    0
label       0
dtype: int64
```

Activity 2.2: Handling Outliers

Boxplots were generated for all features. The Potassium (K) feature showed the most pronounced outliers, confirmed mathematically using the IQR method:

```
Q1 = df['K'].quantile(0.25)
Q3 = df['K'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

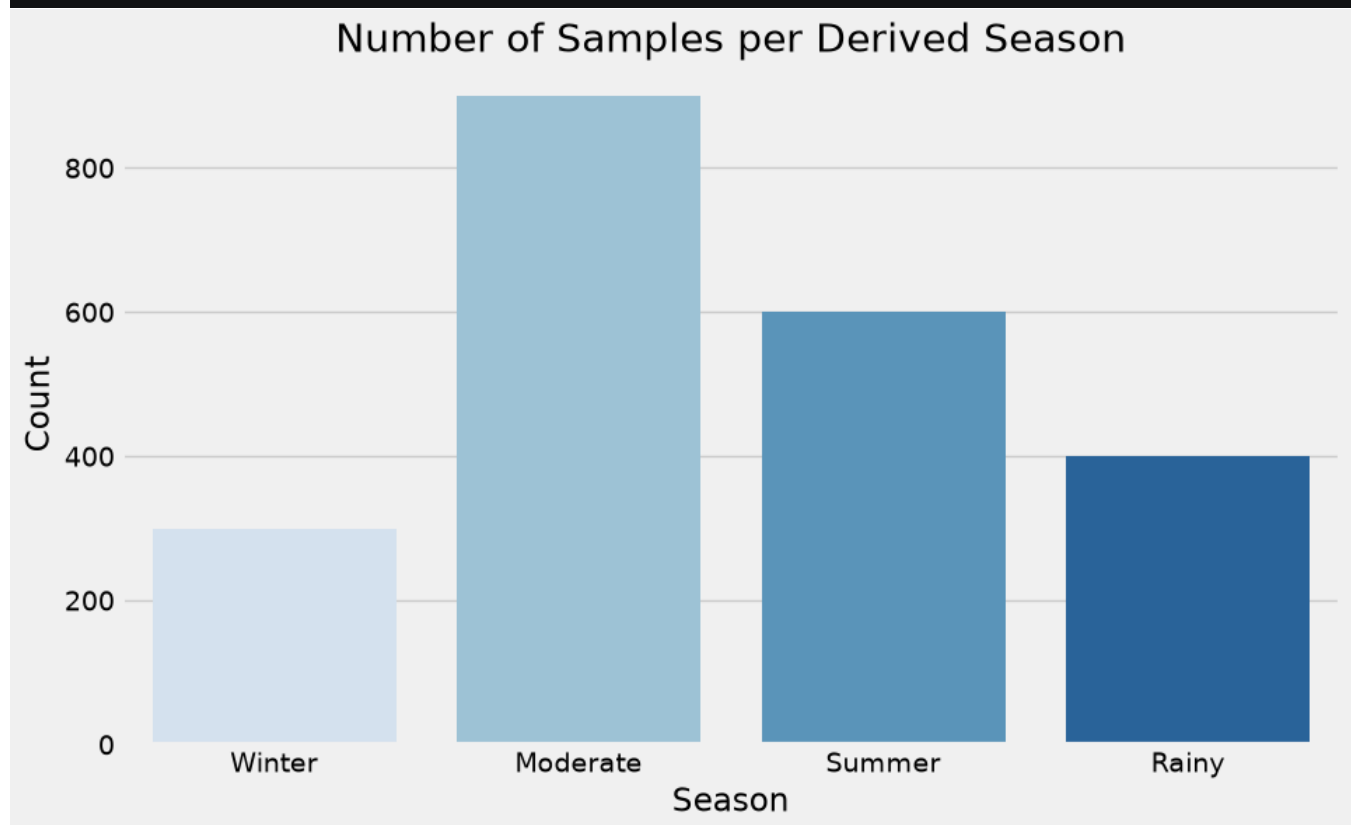
These outliers reflect genuinely high-potassium crops (apple, banana, grapes) rather than data errors, so log transformation was applied to compress the right tail without losing any rows:
`df['K'] = np.log1p(df['K'])`

```
K - Q1: 20.0, Q3: 49.0, IQR: 29.0
K - Lower bound: -23.5, Upper bound: 92.5
Number of outliers in K: 200
```

Activity 2.3: Extracting Seasonal Crops

Each crop was assigned a derived season label (Winter, Summer, Rainy, Moderate) based on its mean temperature and rainfall profile, providing an additional interpretable feature for research-oriented analysis.

```
Winter : ['chickpea', 'kidneybeans', 'pomegranate']
Moderate : ['apple', 'banana', 'cotton', 'grapes', 'lentil', 'maize', 'orange', 'pigeonpeas', 'watermelon']
Summer : ['blackgram', 'mango', 'mothbeans', 'mungbean', 'muskmelon', 'papaya']
Rainy : ['coconut', 'coffee', 'jute', 'rice']
```



Activity 2.4: Train/Test Split

```
X = df[['N','P','K','temperature','humidity','ph','rainfall']]
encoder = LabelEncoder()
y = encoder.fit_transform(df['label'])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

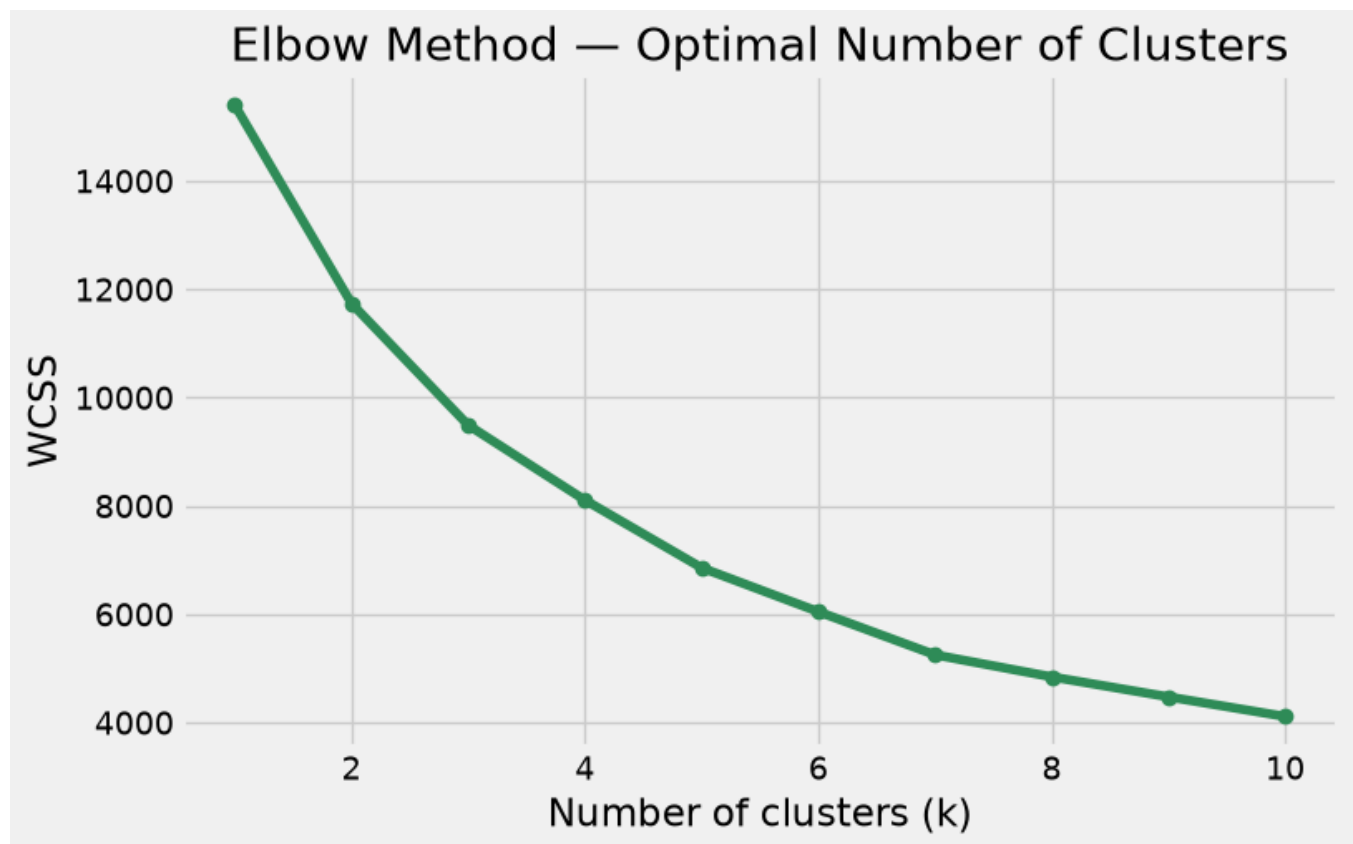
Result: 1,760 training samples, 440 test samples. stratify=y ensures all 22 classes remain proportionally represented in both splits.

```
Train shape: (1760, 7)
Test shape: (440, 7)
```

Milestone 3: Model Building

Activity 3.1: K-Means Clustering

K-Means clustering was applied to discover natural groupings in the soil/climate data. The Elbow Method (WCSS vs. k from 1 to 10) identified k=4 as the optimal number of clusters. The fit_predict() method assigned each row to a cluster, and the dominant crops per cluster were inspected to interpret each group's agro-climatic profile.



```

cluster  label      count
0        chickpea    100
         kidneybeans 100
         mothbeans   100
         lentil      100
         blackgram    100
1        pomegranate 100
         rice        100
         coconut     100
         orange      100
         jute        99
2        banana      100
         muskmelon   100
         watermelon  100
         cotton      100
         maize       82
3        grapes      100
         apple       100
Name: count, dtype: int64

```

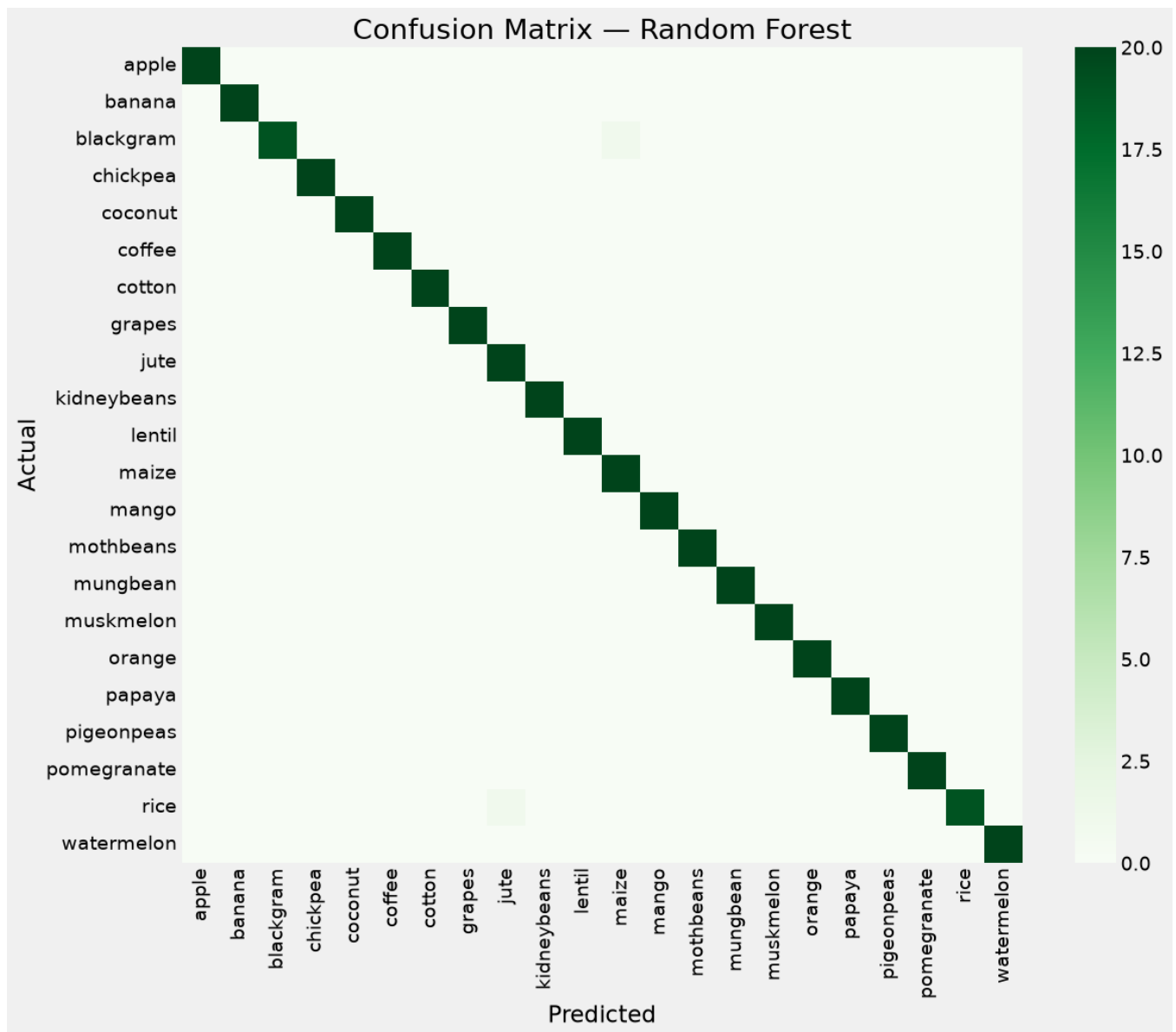
Activity 3.2: Supervised Model Training and Comparison

Four supervised classifiers were trained and evaluated on identical train/test splits using StandardScaler-normalised features:

Model	Accuracy	Precision	Recall	F1-Score
Random Forest (selected)	99.55%	99.57%	99.55%	99.55%
KNN	97.95%	98.04%	97.95%	97.93%
Decision Tree	97.95%	98.06%	97.95%	97.94%
Logistic Regression	97.27%	97.40%	97.27%	97.25%

Random Forest was selected as the final model based on highest accuracy and F1-score across all 22 classes.

CONFUSION MATRIX



Activity 3.3: Saving the Best Model

```
import pickle
with open('../model/model.pkl', 'wb') as f: pickle.dump(best_model, f)
with open('../model/scaler.pkl', 'wb') as f: pickle.dump scaler, f)
with open('../model/label_encoder.pkl', 'wb') as f: pickle.dump(encoder, f)
```

```

import pickle
import os

os.makedirs('../model', exist_ok=True)

with open('../model/model.pkl', 'wb') as f:
    pickle.dump(best_model, f)
with open('../model/scaler.pkl', 'wb') as f:
    pickle.dump(scaler2, f)
with open('../model/label_encoder.pkl', 'wb') as f:
    pickle.dump(encoder, f)

print(f"Saved best model: {best_model_name}")

```

Saved best model: Random Forest

Activity 3.4: Predict the Best Crop Given Parameters

```

def predict_crop(N, P, K, temperature, humidity, ph, rainfall):
    input_data = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
    input_scaled = loaded_scaler.transform(input_data)
    pred_encoded = loaded_model.predict(input_scaled)[0]
    return loaded_encoder.inverse_transform([pred_encoded])[0]

```

```
print(predict_crop(90, 42, 43, 20.88, 82.0, 6.5, 202.9)) # -> rice
```

```

import pickle
import numpy as np

with open('../model/model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)
with open('../model/scaler.pkl', 'rb') as f:
    loaded_scaler = pickle.load(f)
with open('../model/label_encoder.pkl', 'rb') as f:
    loaded_encoder = pickle.load(f)

def predict_crop(N, P, K, temperature, humidity, ph, rainfall):
    input_data = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
    input_scaled = loaded_scaler.transform(input_data)
    pred_encoded = loaded_model.predict(input_scaled)[0]
    return loaded_encoder.inverse_transform([pred_encoded])[0]

print("Recommended crop:", predict_crop(90, 42, 43, 20.879744, 82.002744, 6.502985, 202.935536))

```

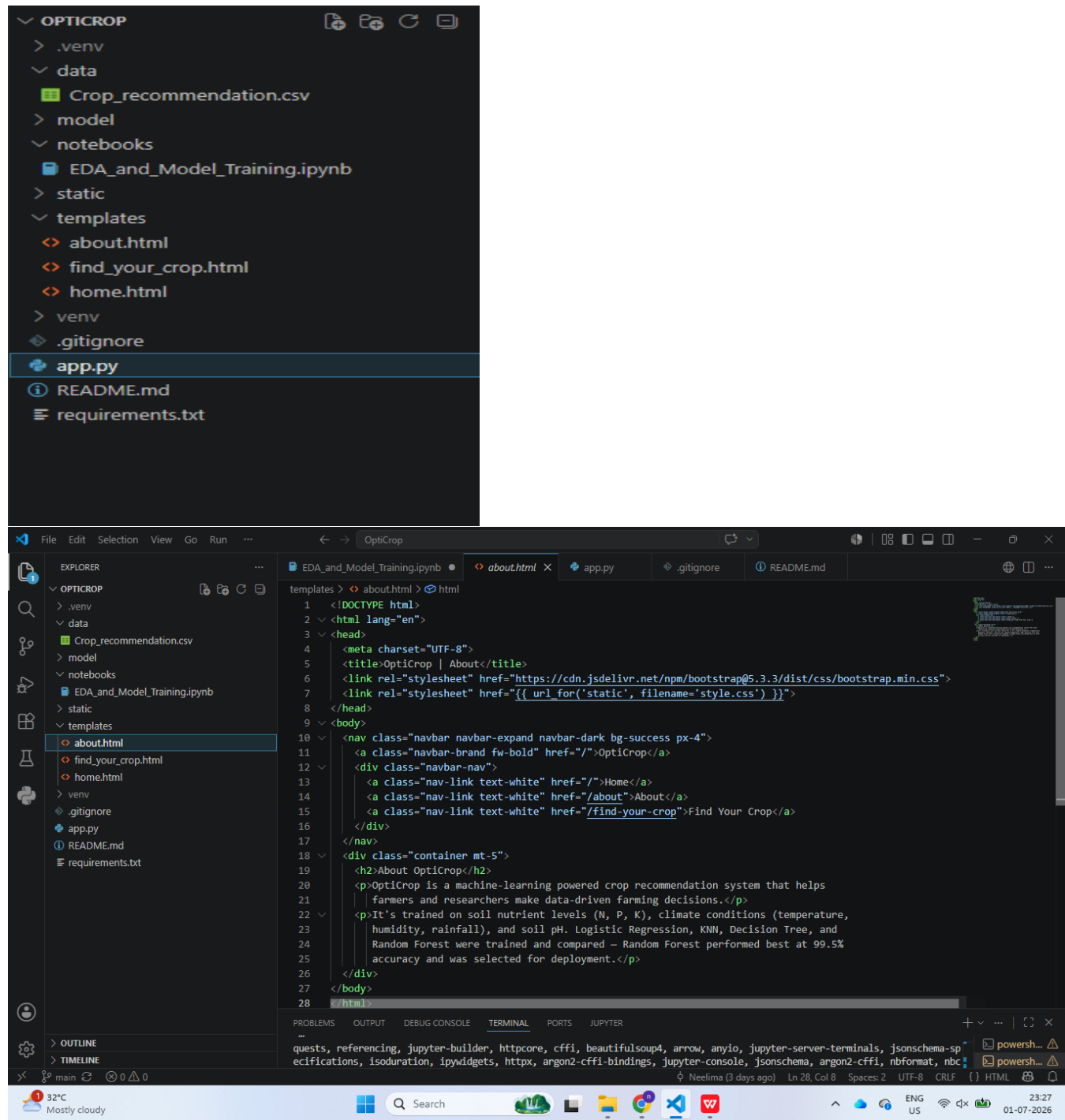
Recommended crop: rice

Milestone 4: Application Building

Activity 4.1: Building HTML Pages

Three Flask templates were created in the templates/ folder:

- **home.html:** Project introduction, tagline, and a prominent 'Find Your Crop' call-to-action button. Navigation bar with links to all three pages.
- **about.html:** Project background, methodology overview, and model performance summary.
- **find_your_crop.html:** Input form collecting N, P, K, temperature, humidity, pH, and rainfall. On POST, the same page re-renders with the prediction displayed below the form.



Activity 4.2: Building the Python Backend (app.py)

```
from flask import Flask, render_template, request
import pickle, numpy as np
```

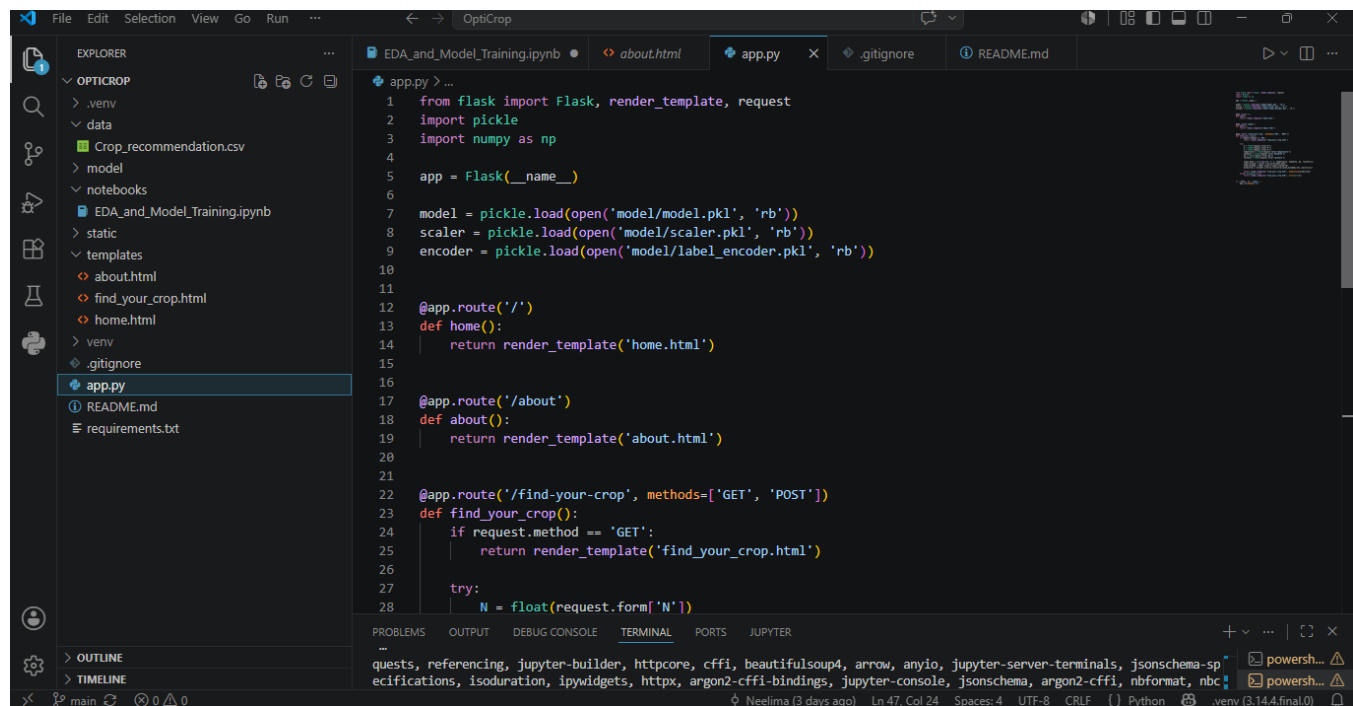
```
app = Flask(__name__)
model = pickle.load(open('model/model.pkl', 'rb'))
scaler = pickle.load(open('model/scaler.pkl', 'rb'))
encoder = pickle.load(open('model/label_encoder.pkl', 'rb'))
```

```
@app.route('/')
def home(): return render_template('home.html')
```

```
@app.route('/about')
def about(): return render_template('about.html')
```

```
@app.route('/find-your-crop', methods=['GET', 'POST'])
def find_your_crop():
    if request.method == 'GET': return render_template('find_your_crop.html')
    values = [float(request.form[f]) for f in
               ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]
    X = scaler.transform([values])
    crop = encoder.inverse_transform(model.predict(X))[0].capitalize()
    return render_template('find_your_crop.html', prediction=crop)
```

```
if __name__ == '__main__': app.run(debug=True)
```



Activity 4.3: Run and Test the Application

- Open a terminal in the OptiCrop root folder.
- Activate the virtual environment: `venv\Scripts\activate`
- Run the app: `python app.py`
- Open `http://127.0.0.1:5000` in the browser.
- Test all three pages and confirm the prediction returns 'Rice' for N=90, P=42, K=43, temperature=20.88, humidity=82.0, pH=6.5, rainfall=202.9.

Find Your Crop

Enter your soil and climate conditions to get a crop recommendation.

Nitrogen (N)

Phosphorous (P)

Potassium (K)

Temperature (°C)

Humidity (%)

Soil pH

Rainfall (mm)

Predict

Find Your Crop

Enter your soil and climate conditions to get a crop recommendation.

Nitrogen (N)

Phosphorous (P)

Potassium (K)

Temperature (°C)

Humidity (%)

Soil pH

Rainfall (mm)

Predict

Recommended crop: **Mothbeans**

IN VS CODE

```
print("Recommended crop:", predict_crop(20, 60, 20, 28, 50, 6.0, 50))
```

```
Recommended crop: mothbeans
```

RUN APPLICATION

```
(.venv) PS C:\Users\ADMIN\OptiCrop> python app.py
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 711-102-982
127.0.0.1 - - [01/Jul/2026 12:30:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 12:30:42] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [01/Jul/2026 12:30:42] "GET /favicon.ico HTTP/1.1" 404 -
```

Usage Instructions

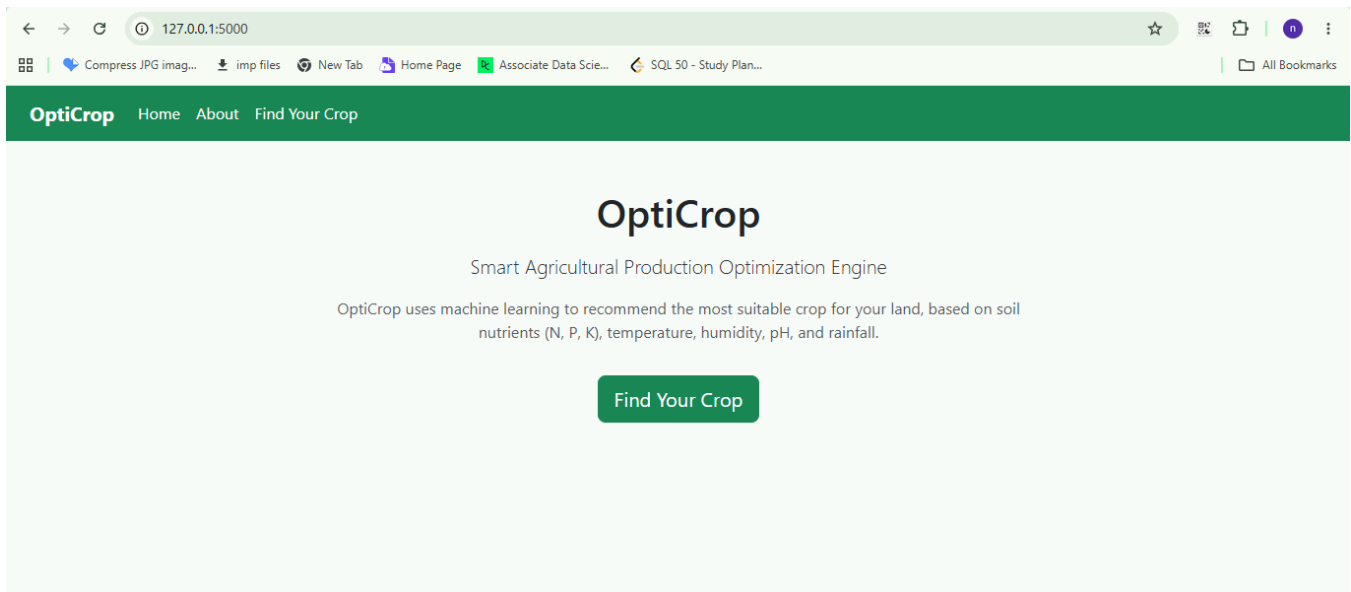
Installation Steps

1. Clone the repository: `git clone https://github.com/neelimavana/OptiCrop.git`
2. Navigate into the project folder: `cd OptiCrop`
3. Create a virtual environment: `python -m venv venv`
4. Activate the environment: `venvScriptsactivate` (Windows)
5. Install dependencies: `pip install -r requirements.txt`
6. Run the application: `python app.py`
7. Open `http://127.0.0.1:5000` in your browser.

Exploring the Web Application Pages

Home Page

The home page is the entry point of the OptiCrop application. It features a clean navigation bar with links to Home, About, and Find Your Crop. The main section displays the project title, a short tagline ('Smart Agricultural Production Optimization Engine'), a brief description of what OptiCrop does, and a prominent green 'Find Your Crop' button that navigates directly to the prediction form.



About Page

The About page explains the project's purpose, the dataset used (2,200 samples, 22 crop classes, 7 soil and climate features), and the methodology — EDA, pre-processing, multi-model comparison, and Random Forest selection at 99.55% accuracy. It gives any user, including a non-technical farmer, enough context to trust the recommendation the system provides.

OptiCrop

[Home](#)[About](#)[Find Your Crop](#)

Find Your Crop

Enter your soil and climate conditions to get a crop recommendation.

Nitrogen (N)

20

Phosphorous (P)

60

Potassium (K)

20

Temperature (°C)

28

Humidity (%)

50

Soil pH

6.0

Rainfall (mm)

50

Predict

Find Your Crop Page

The Find Your Crop page is the core feature of the application. It presents a form with seven clearly labelled numeric input fields: Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, Soil pH, and Rainfall. On form submission, the backend scales the inputs, runs the Random Forest model, decodes the prediction, and re-renders the same page with the recommended crop name displayed prominently below the form. Input validation ensures non-numeric or empty submissions return a clean error message.

OptiCrop

[Home](#)[About](#)[Find Your Crop](#)

Find Your Crop

Enter your soil and climate conditions to get a crop recommendation.

Nitrogen (N)

Phosphorous (P)

Potassium (K)

Temperature (°C)

Humidity (%)

Soil pH

Rainfall (mm)

Predict

Recommended crop: Mothbeans

Milestone 5: Conclusion

OptiCrop is a machine-learning powered smart agricultural recommendation system that demonstrates a complete, reproducible ML lifecycle — from raw agricultural data through exploratory data analysis, pre-processing, multi-algorithm model comparison, and Flask-based web deployment — packaged into an accessible tool for farmers, researchers, and policymakers alike.

The system analysed 2,200 samples across 22 balanced crop classes using 7 soil and climate features. After comparing Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest on identical train/test splits, Random Forest was selected as the final deployed model, achieving 99.55% test accuracy with matching Precision, Recall, and F1-Score — confirming strong generalisation rather than overfitting.

The resulting three-page Flask application (Home, About, Find Your Crop) allows any user with basic soil and climate measurements to get an instant, model-backed crop recommendation with no ML expertise required. The full pipeline is reproducible: dataset, trained model artefacts, training notebook, and application code are all publicly available at <https://github.com/neelimavana/OptiCrop>.

Looking ahead, OptiCrop can be extended with real-time weather API integration (auto-filling temperature, humidity, and rainfall for a given location), fertilizer and irrigation guidance alongside crop recommendations, a larger regionally-specific dataset covering India's diverse agro-climatic zones, and cloud deployment to make the application publicly accessible beyond local hosting.